

Techastra'26 — Code Rescue

Debugging Competition Question Paper

ROUND 1 - BUG HUNT

10 Questions - Exactly 2 Errors Each - Advanced

Q1. Student Average

1. WORK ORDER DESCRIPTION

The following function is designed to calculate the average of all marks in a student's mark list. However, the implementation contains two logic errors that cause the calculated average to be incorrect.

2. EXPECTED BEHAVIOR / CONTRACT

The function must include every mark in the calculation and return the correct arithmetic average.

3. I/O SPECIFICATION

Input Format:

A list of integers representing student marks.

Output Format:

The calculated average as a numeric value.

4. OPERATIONAL CONSTRAINTS

The marks list contains valid integers.

All marks must be considered.

The function must not modify the original list.

5. VISIBLE VERIFICATION BASELINE

Sample Input

[78, 85, 92, 88]

Expected Output

Average: 85.75

6. PROGRAM

```
def average(marks):  
    total = 0  
    for i in range(len(marks) - 1):  
        total += marks[i]  
    return total / len(marks)
```

```
marks = [78, 85, 92, 88]  
print("Average:", average(marks))
```

Q2. Prime Number Checker

1. WORK ORDER DESCRIPTION

The following function determines whether a given integer is prime. The implementation contains two errors in its divisor-checking logic.

2. EXPECTED BEHAVIOR / CONTRACT

The function must return True only when the supplied number is prime and False otherwise.

3. I/O SPECIFICATION

Input Format:

One integer n.

Output Format:

Boolean value: True or False.

4. OPERATIONAL CONSTRAINTS

Numbers less than 2 are not prime.

Divisors only need to be checked up to the square root of the number.

The original number must not be modified.

5. VISIBLE VERIFICATION BASELINE

Sample Input

49

Expected Output

False

6. PROGRAM

```
def is_prime(n):
    if n < 2:
        return False

    for i in range(2, int(n ** 0.5)):
        if n % i == 0:
            return False

    return True

number = 49
print(is_prime(number))
```

Q3. Reverse a Number

1. WORK ORDER DESCRIPTION

The following function is intended to reverse the digits of an integer. It contains two errors in digit extraction and number reduction.

2. EXPECTED BEHAVIOR / CONTRACT

The function must return the digits of the supplied positive integer in reverse order.

3. I/O SPECIFICATION

Input Format:

One positive integer.

Output Format:

The reversed integer.

4. OPERATIONAL CONSTRAINTS

Input is a valid positive integer.

Each digit must be processed individually.

5. VISIBLE VERIFICATION BASELINE

Sample Input

1234

Expected Output

4321

6. PROGRAM

```
def reverse_number(n):
    rev = 0
    while n > 0:
        digit = n // 10
        rev = rev * 10 + digit
        n = n % 10
    return rev

print(reverse_number(1234))
```

Q4. Find Maximum

1. WORK ORDER DESCRIPTION

The following function is designed to find the largest value in a list of integers. Its initialization and comparison logic contain two errors.

2. EXPECTED BEHAVIOR / CONTRACT

The function must return the maximum value, including when all numbers are negative.

3. I/O SPECIFICATION

Input Format:

A list of integers.

Output Format:

The maximum integer.

4. OPERATIONAL CONSTRAINTS

The list contains at least one integer.

Negative values must be handled correctly.

5. VISIBLE VERIFICATION BASELINE

Sample Input

[-12, -5, -20, -3]

Expected Output

-3

6. PROGRAM

```
def find_max(numbers):  
    maximum = 0  
  
    for num in numbers:  
        if num < maximum:  
            maximum = num  
  
    return maximum  
  
numbers = [-12, -5, -20, -3]  
print(find_max(numbers))
```

Q5. Fibonacci

1. WORK ORDER DESCRIPTION

The following function generates the first n Fibonacci numbers. Two errors cause the Fibonacci sequence to be calculated incorrectly.

2. EXPECTED BEHAVIOR / CONTRACT

The function must generate the Fibonacci sequence beginning with 0, 1.

3. I/O SPECIFICATION

Input Format:

One positive integer n.

Output Format:

The first n Fibonacci numbers.

4. OPERATIONAL CONSTRAINTS

The sequence must begin with 0 1.

Exactly n values must be displayed.

5. VISIBLE VERIFICATION BASELINE

Sample Input

6

Expected Output

0 1 1 2 3 5

6. PROGRAM

```
def fibonacci(n):
    a, b = 0, 1

    for _ in range(n):
        print(a, end=" ")
        a = a + b
        b = a

    fibonacci(6)
```

Q6. Palindrome Checker

1. WORK ORDER DESCRIPTION

The following function checks whether a string reads the same forwards and backwards. The comparison logic contains an incorrect return value.

2. EXPECTED BEHAVIOR / CONTRACT

The function must return True when the input is a palindrome and False otherwise.

3. I/O SPECIFICATION

Input Format:

One string.

Output Format:

Boolean value.

4. OPERATIONAL CONSTRAINTS

Comparison must be case-insensitive.

The original string should not be modified outside the function.

5. VISIBLE VERIFICATION BASELINE

Sample Input

Level

Expected Output

True

6. PROGRAM

```
def is_palindrome(text):
    text = text.lower()
    reversed_text = text[::-1]

    if text == reversed_text:
        return False
    return True

print(is_palindrome("Level"))
```

Q7. Dictionary Counter

1. WORK ORDER DESCRIPTION

The following function counts how many times each word occurs in a list. The dictionary initialization and increment logic contain two errors.

2. EXPECTED BEHAVIOR / CONTRACT

Each unique word must be stored with its correct frequency.

3. I/O SPECIFICATION

Input Format:

A list of strings.

Output Format:

A dictionary containing word-frequency pairs.

4. OPERATIONAL CONSTRAINTS

Every occurrence must be counted.

Words are case-sensitive.

5. VISIBLE VERIFICATION BASELINE

Sample Input

```
["python", "java", "python", "c"]
```

Expected Output

```
{'python': 2, 'java': 1, 'c': 1}
```

6. PROGRAM

```
def count_words(words):
    count = {}

    for word in words:
        if word in count:
            count[word] = 1
        else:
            count[word] += 1

    return count
```

Q8. Second Largest

1. WORK ORDER DESCRIPTION

The following function attempts to determine the second-largest value in a list. Its handling of duplicate maximum values causes an incorrect result.

2. EXPECTED BEHAVIOR / CONTRACT

The function must return the second-largest distinct value.

3. I/O SPECIFICATION

Input Format:

A list of integers.

Output Format:

Second-largest distinct integer.

4. OPERATIONAL CONSTRAINTS

Duplicate maximum values may exist.

The second-largest value must be distinct from the largest.

5. VISIBLE VERIFICATION BASELINE

Sample Input

```
[10, 25, 25, 40, 15]
```

Expected Output

```
25
```

6. PROGRAM

```
def second_largest(numbers):
    largest = max(numbers)
    numbers.remove(largest)
    return max(numbers)

numbers = [10, 25, 25, 40, 15]
print(second_largest(numbers))
```

Q9. Recursive Factorial

1. WORK ORDER DESCRIPTION

The following recursive function is intended to calculate the factorial of an integer. The base case and recursive progression contain two errors.

2. EXPECTED BEHAVIOR / CONTRACT

The function must return $n!$ for a valid non-negative integer.

3. I/O SPECIFICATION

Input Format:

One integer n .

Output Format:

Factorial of n .

4. OPERATIONAL CONSTRAINTS

Recursive implementation must be used.

The recursion must terminate correctly.

5. VISIBLE VERIFICATION BASELINE

Sample Input

5

Expected Output

120

6. PROGRAM

```
def factorial(n):  
    if n == 1:  
        return 1  
  
    return n * factorial(n + 1)  
  
print(factorial(5))
```

Q10. List Filtering

1. WORK ORDER DESCRIPTION

The following list comprehension is intended to select odd numbers greater than 20. The filtering condition contains errors.

2. EXPECTED BEHAVIOR / CONTRACT

Only numbers greater than 20 and odd should be selected.

3. I/O SPECIFICATION

Input Format:

A list of integers.

Output Format:

A filtered list.

4. OPERATIONAL CONSTRAINTS

Values must be greater than 20.

Values must be odd.

5. VISIBLE VERIFICATION BASELINE

Sample Input

[10, 15, 20, 25, 30, 35]

Expected Output

[25, 35]

6. PROGRAM

```
numbers = [10, 15, 20, 25, 30, 35]

result = [n for n in numbers if n % 2 == 1 and n > 20]

print(result)
```

ROUND 2 - LOGIC BREAKER

10 Questions - Exactly 5 Errors Each - Advanced The same six-section format will be used for every question.

Q1. Student Marks Analyzer

1. WORK ORDER DESCRIPTION

The program calculates a student's total marks, average and performance result. Multiple logical errors cause incorrect totals, averages and result classification.

2. EXPECTED BEHAVIOR / CONTRACT

The program must calculate the complete total, correct average and assign the appropriate result category according to the grading rules.

3. I/O SPECIFICATION

Input: Student marks [78, 92, 65, 88, 74]

Output:

Total: 397

Average: 79.4

Result: DISTINCTION

4. OPERATIONAL CONSTRAINTS

Every mark must be included.

Average must be calculated from all marks.

Grade/result conditions must be evaluated in the correct order.

No marks may be modified.

5. VISIBLE VERIFICATION BASELINE

Sample Input:

[78,92,65,88,74]

Expected Output:

397, 79.4, DISTINCTION

6. PROGRAM

```
marks = [78, 92, 65, 88, 74]

total = 0

for i in range(len(marks) - 1):
    total += marks[i]

average = total / len(marks)

if average > 40:
    result = "FAIL"
elif average >= 75:
    result = "DISTINCTION"
else:
    result = "PASS"

print("Total:", total)
print("Average:", average)
print("Result:", result)
```

Q2. Electricity Bill

1. WORK ORDER DESCRIPTION

The program calculates an electricity bill using multiple consumption slabs. Errors in slab calculation and billing logic produce an incorrect final bill.

2. EXPECTED BEHAVIOR / CONTRACT

The program must calculate the bill according to the defined unit slabs.

3. I/O SPECIFICATION

Input: 275 units

Output:

Bill: 825

4. OPERATIONAL CONSTRAINTS

Units are valid non-negative integers.

Each slab must be calculated independently.

Previous slab charges must be included correctly.

5. VISIBLE VERIFICATION BASELINE

Sample Input:

275 units

Expected Output:

825

6. PROGRAM

```
units = 275

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
elif units <= 300:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
else:
    bill = 100 * 2 + 100 * 3 + 100 * 5 + (units - 300) * 7

print("Bill:", bill)
```

Q3. Shopping Discount

1. WORK ORDER DESCRIPTION

The program calculates a shopping subtotal, determines a discount based on the subtotal and produces the final payable amount. Errors in discount selection and final amount calculation produce an incorrect bill.

2. EXPECTED BEHAVIOR / CONTRACT

The program must include all item prices, apply the correct discount slab and subtract the discount from the subtotal.

3. I/O SPECIFICATION

Input: 1200, 850, 500, 300

Output:

Subtotal: 2850

Discount: 285

Final: 2565

4. OPERATIONAL CONSTRAINTS

All item prices are valid.

Discount must be calculated from the subtotal.

Discount must reduce the final payable amount.

5. VISIBLE VERIFICATION BASELINE

[1200,850,500,300]

2850 / 285 / 2565

6. PROGRAM

```
items = [1200, 850, 500, 300]

subtotal = sum(items)

if subtotal > 3000:
    discount = subtotal * 0.20
elif subtotal > 2000:
    discount = subtotal * 0.10
else:
    discount = 0

final_amount = subtotal + discount

print("Subtotal:", subtotal)
print("Discount:", discount)
print("Final:", final_amount)
```

Q4. Number Frequency

1. WORK ORDER DESCRIPTION

The program determines the frequency of each number in an integer list and identifies the number occurring most frequently.

2. EXPECTED BEHAVIOR / CONTRACT

The program must correctly count every occurrence and display the number with the highest frequency.

3. I/O SPECIFICATION

Input:

[2, 3, 2, 5, 3, 2, 7]

Output:

2 3

4. OPERATIONAL CONSTRAINTS

Every occurrence must be counted.

The highest frequency must be calculated dynamically.

5. VISIBLE VERIFICATION BASELINE

[2,3,2,5,3,2,7]

2 3

6. PROGRAM

```
numbers = [2, 3, 2, 5, 3, 2, 7]

frequency = {}

for n in numbers:
    if n not in frequency:
        frequency[n] = 0
    frequency[n] += 1

highest = max(frequency.values())

for n, count in frequency.items():
    if count == highest:
        print(n, count)
```

Q5. Armstrong Number

1. WORK ORDER DESCRIPTION

The program determines whether a number is an Armstrong number by raising each digit to the power of the number of digits.

2. EXPECTED BEHAVIOR / CONTRACT

The program must correctly determine whether the sum of powered digits equals the original number.

3. I/O SPECIFICATION

Input: 153

Output:

Armstrong

4. OPERATIONAL CONSTRAINTS

Input must be a valid positive integer.

Every digit must be processed.

5. VISIBLE VERIFICATION BASELINE

153

Armstrong

6. PROGRAM

```
number = 153
temp = number
digits = len(str(number))
total = 0

while temp > 0:
    digit = temp % 10
    total += digit ** digits
    temp = temp // 10

if total == number:
    print("Armstrong")
else:
    print("Not Armstrong")
```

Q6. Matrix Diagonal Sum

1. WORK ORDER DESCRIPTION

The program calculates the sum of the main diagonal and secondary diagonal of a square matrix.

2. EXPECTED BEHAVIOR / CONTRACT

Both diagonal sums must be calculated using the correct matrix indexes.

3. I/O SPECIFICATION

Input:

1 2 3

4 5 6

7 8 9

Output:

Main: 15

Secondary: 15

4. OPERATIONAL CONSTRAINTS

Matrix must be square.

Main diagonal uses matching row/column indexes.

Secondary diagonal must be calculated from the opposite direction.

5. VISIBLE VERIFICATION BASELINE

3 x 3 matrix

Main: 15, Secondary: 15

6. PROGRAM

```

matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

main = 0
secondary = 0

for i in range(len(matrix)):
    main += matrix[i][i]
    secondary += matrix[i][len(matrix) - i - 1]

print("Main:", main)
print("Secondary:", secondary)

```

Q7. ATM Withdrawal

1. WORK ORDER DESCRIPTION

The program processes an ATM withdrawal request and verifies balance and withdrawal amount validity.

2. EXPECTED BEHAVIOR / CONTRACT

A valid withdrawal must decrease the balance and display the remaining balance.

3. I/O SPECIFICATION

Input:

Balance: 10000

Amount: 3500

Output:

Withdrawal successful

Remaining: 6500

4. OPERATIONAL CONSTRAINTS

Withdrawal cannot exceed available balance.

Withdrawal amount must be a valid multiple of 100.

Balance must decrease after a successful withdrawal.

5. VISIBLE VERIFICATION BASELINE

10000, 3500

6500 remaining

6. PROGRAM

```

balance = 10000
amount = 3500

if amount > balance:
    print("Insufficient balance")
elif amount % 100 != 0:
    print("Invalid amount")
else:
    balance -= amount
    print("Withdrawal successful")
    print("Remaining:", balance)

```

Q8. Student Grade

1. WORK ORDER DESCRIPTION

The program calculates total marks, average and grade for a student.

2. EXPECTED BEHAVIOR / CONTRACT

The program must calculate the correct total and average and assign the appropriate grade.

3. I/O SPECIFICATION

Input:

[85, 78, 92, 88, 80]

Output:

423 84.6 A

4. OPERATIONAL CONSTRAINTS

All marks must be included.

Grade boundaries must be evaluated from highest to lowest.

Average must be calculated using floating-point division.

5. VISIBLE VERIFICATION BASELINE

[85,78,92,88,80]

423 84.6 A

6. PROGRAM

```
marks = [85, 78, 92, 88, 80]
```

```
total = sum(marks)
average = total / len(marks)
```

```
if average >= 90:
    grade = "A+"
elif average >= 80:
    grade = "A"
elif average >= 70:
    grade = "B"
elif average >= 60:
    grade = "C"
else:
    grade = "F"
```

```
print(total, average, grade)
```

Q9. Search an Array

1. WORK ORDER DESCRIPTION

The program searches for a target value inside an integer array and reports whether it exists.

2. EXPECTED BEHAVIOR / CONTRACT

The program must display Found when the target exists and Not Found otherwise.

3. I/O SPECIFICATION

Input:

Numbers: [12, 45, 67, 23, 89]

Target: 67

Output:

Found

4. OPERATIONAL CONSTRAINTS

Search must inspect the provided array.

Once the target is found, unnecessary searching should stop.

5. VISIBLE VERIFICATION BASELINE

67

Found

6. PROGRAM

```
numbers = [12, 45, 67, 23, 89]
```

```

target = 67
found = False

for n in numbers:
    if n == target:
        found = True
        break

if found:
    print("Found")
else:
    print("Not Found")

```

Q10. Password Validation

1. WORK ORDER DESCRIPTION

The program validates a password by checking for at least one digit, one uppercase letter and one special character.

2. EXPECTED BEHAVIOR / CONTRACT

The password must be declared valid only when all required character categories are present.

3. I/O SPECIFICATION

Input:

Python@123

Output:

Valid Password

4. OPERATIONAL CONSTRAINTS

The password must contain:

At least one digit.

At least one uppercase letter.

At least one supported special character.

5. VISIBLE VERIFICATION BASELINE

Python@123

Valid Password

6. PROGRAM

```

password = "Python@123"

has_digit = False
has_upper = False
has_special = False

for ch in password:
    if ch.isdigit():
        has_digit = True
    if ch.isupper():
        has_upper = True
    if ch in "@#%":
        has_special = True

if has_digit and has_upper and has_special:
    print("Valid Password")
else:
    print("Invalid Password")

```

ROUND 3 - CODE RESCUE

10 Questions - Large Integrated Programs - Moderate For Round 3, each program follows the same six-section structure.

Q1. Student Marks Management System

1. WORK ORDER DESCRIPTION

The following Student Marks Management System is responsible for adding students, calculating their total marks and average, determining their grade, and identifying the student with the highest total.

The program contains multiple errors distributed across its functions. Locate and repair all 12 errors without changing the intended functionality.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Correctly store each student's information.

Calculate the complete total of all marks.

Calculate the correct average.

Assign grades according to the defined grade ranges.

Display every student's information.

Identify the student with the highest total.

3. I/O SPECIFICATION

Input Format:

Student name and list of marks provided through the program.

Output Format:

Name:

Total:

Average:

Grade:

4. OPERATIONAL CONSTRAINTS

All marks are valid integers.

Every mark must be included.

Grade ranges must be checked from highest to lowest.

The highest scorer must be determined from total marks.

5. VISIBLE VERIFICATION BASELINE

Arun [78,85,92,88,50]

Total 393, Average 78.6

Bala [65,72,80,75,68]

Total 360, Average 72.0

Chris [90,88,95,91,85]

Total 449, Average 89.8

Highest Scorer: Chris

6. PROGRAM

```

students = []

def add_student(name, marks):
    student = {
        "name": name,
        "marks": marks
    }
    students.append(student)

def calculate_total(marks):
    total = 0

    for i in range(len(marks) - 1):
        total += marks[i]

    return total

def calculate_average(marks):
    total = calculate_total(marks)
    return total / len(marks) - 1

def calculate_grade(average):
    if average >= 80:
        return "A"
    elif average >= 90:
        return "A+"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    else:
        return "F"

def display_students():
    highest = 0
    highest_student = ""

    for student in students:
        total = calculate_total(student["marks"])
        average = calculate_average(student["marks"])
        grade = calculate_grade(average)

        if total < highest:
            highest = total
            highest_student = student["name"]

        print("Name:", student["name"])
        print("Total:", total)
        print("Average:", round(average, 2))
        print("Grade:", grade)
        print()

add_student("Arun", [78, 85, 92, 88, 50])
add_student("Bala", [65, 72, 80, 75, 68])
add_student("Chris", [90, 88, 95, 91, 85])

display_students()

```

Q2. Electricity Bill Management System

1. WORK ORDER DESCRIPTION

The following Electricity Bill Management System calculates electricity charges for multiple customers using a slab-based billing structure. The program contains 11 errors affecting bill calculation and final billing behavior.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Correctly determine the applicable electricity slab.

Calculate charges for every consumed unit.

Return the calculated bill.

Calculate surcharge when applicable.

Display the correct final bill.

3. I/O SPECIFICATION

Input Format:

Customer name

Units consumed

Output Format:

Customer:

Units:

Bill:

Surcharge:

Final Bill:

4. OPERATIONAL CONSTRAINTS

Units are non-negative integers.

Slab calculations must be cumulative.

Surcharge must be added to the bill, not incorrectly deducted.

Every customer must be processed independently.

5. VISIBLE VERIFICATION BASELINE

Customer

Units

Arun

75

Bala

150

Chris

275

David

350

6. PROGRAM

```
def calculate_bill(units):  
    if units < 100:  
        bill = units * 2  
  
    elif units <= 200:  
        bill = 100 * 2 + (units - 100) * 3  
  
    elif units <= 300:  
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5  
  
    else:  
        bill = 100 * 2 + 100 * 3 + 100 * 5 + (units - 300) * 7  
  
    return bill  
  
def display_bill(name, units):  
    bill = calculate_bill(units)  
  
    if bill > 1000:  
        surcharge = bill * 0.10  
    else:  
        surcharge = 0  
  
    final_bill = bill - surcharge  
  
    print("Customer:", name)  
    print("Units:", units)  
    print("Bill:", bill)  
    print("Surcharge:", surcharge)  
    print("Final Bill:", final_bill)  
  
customers = [
```

```

        ("Arun", 75),
        ("Bala", 150),
        ("Chris", 275),
        ("David", 350)
    ]

    for customer in customers:
        display_bill(customer[0], customer[1])

```

Q3. Shopping and Discount System

1. WORK ORDER DESCRIPTION

The following Shopping and Discount System calculates the subtotal of a customer's cart, applies a discount based on the subtotal and produces the final payable amount. The program contains 12 errors, including incorrect quantity handling and discount calculation.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Calculate each product's cost using price x quantity.

Calculate the complete subtotal.

Apply the appropriate discount percentage.

Subtract the discount from the subtotal.

Display the final amount.

3. I/O SPECIFICATION

Input Format:

Product name and quantity.

Output Format:

Subtotal:

Discount:

Final Amount:

4. OPERATIONAL CONSTRAINTS

Product names must exist in the product catalogue.

Quantities must be included.

Discount must be calculated from subtotal.

Final amount must not increase because of a discount.

5. VISIBLE VERIFICATION BASELINE

Laptop 1

Mouse 2

Keyboard 1

Headphones 1

Correct Subtotal: 60600

Discount: 6060

Final Amount: 54540

6. PROGRAM

```

products = {
    "Laptop": 55000,
    "Mouse": 800,
    "Keyboard": 1500,
    "Headphones": 2500
}

```

```

def calculate_subtotal(cart):
    subtotal = 0

    for product, quantity in cart.items():
        subtotal += products[product]

    return subtotal

def calculate_discount(subtotal):
    if subtotal >= 50000:
        return subtotal * 0.10
    elif subtotal >= 30000:
        return subtotal * 0.15
    elif subtotal >= 10000:
        return subtotal * 0.05
    else:
        return 0

def generate_bill(cart):
    subtotal = calculate_subtotal(cart)
    discount = calculate_discount(subtotal)

    final_amount = subtotal + discount

    print("Subtotal:", subtotal)
    print("Discount:", discount)
    print("Final Amount:", final_amount)

cart = {
    "Laptop": 1,
    "Mouse": 2,
    "Keyboard": 1,
    "Headphones": 1
}

generate_bill(cart)

```

Q4. ATM Banking System

1. WORK ORDER DESCRIPTION

The following ATM Banking System provides four operations: checking balance, depositing money, withdrawing money and exiting the application. The program contains 15 errors affecting variable scope, arithmetic operations, validation and menu selection.

2. EXPECTED BEHAVIOR / CONTRACT

All four ATM operations must function correctly.

3. I/O SPECIFICATION

Input Format:

1 -> Balance

2 -> Deposit

3 -> Withdraw

4 -> Exit

Output Format:

Appropriate transaction message and updated balance.

4. OPERATIONAL CONSTRAINTS

Deposit must increase balance.

Withdrawal must decrease balance.

Withdrawal cannot exceed available balance.

Withdrawal must be a valid multiple of 100.

Menu options must be correctly recognized.

5. VISIBLE VERIFICATION BASELINE

Operation | Sample Input | Expected Result

Balance | 1 | 10000

Deposit | 2 -> 2000 | 12000

Withdraw | 3 -> 3500 | 8500

Exit | 4 | Program terminates

6. PROGRAM

```
balance = 10000

def check_balance():
    print("Current Balance:", balance)

def deposit(amount):
    if amount < 0:
        print("Invalid deposit")
        return

    balance = balance + amount
    print("Deposit successful")

def withdraw(amount):
    global balance

    if amount > balance:
        print("Insufficient balance")
        return

    if amount % 100 != 0:
        print("Invalid withdrawal amount")
        return

    balance -= amount

    print("Withdrawal successful")

def menu():
    while True:
        print("\n1. Balance")
        print("2. Deposit")
        print("3. Withdraw")
        print("4. Exit")

        choice = input("Enter choice: ")

        if choice == 1:
            check_balance()

        elif choice == 2:
            amount = int(input("Enter amount: "))
            deposit(amount)

        elif choice == 3:
            amount = int(input("Enter amount: "))
            withdraw(amount)

        elif choice == 4:
            print("Thank you")
            break

        else:
            print("Invalid choice")

menu()
```

Q5. Employee Salary Management

1. WORK ORDER DESCRIPTION

The following Employee Salary Management System calculates HRA, DA, gross salary, tax and net salary for multiple employees. It also needs to identify the employee receiving the highest net salary. The program contains 13 errors.

2. EXPECTED BEHAVIOR / CONTRACT

The system must correctly:

Calculate HRA.

Calculate DA.

Calculate gross salary.

Apply the appropriate tax.

Calculate net salary.

Identify the highest-paid employee.

3. I/O SPECIFICATION

Input Format:

Employee name

Basic salary

Output Format:

Employee:

Basic:

Gross:

Tax:

Net:

4. OPERATIONAL CONSTRAINTS

HRA and DA must be calculated from basic salary.

Tax must be deducted from gross salary.

Highest-paid employee must be determined using net salary.

5. VISIBLE VERIFICATION BASELINE

Employee | Basic

Arun | Rs. 30,000

Bala | Rs. 40,000

Chris | Rs. 50,000

Highest-paid employee: Chris

6. PROGRAM

```
employees = [  
    {"name": "Arun", "basic": 30000},  
    {"name": "Bala", "basic": 40000},  
    {"name": "Chris", "basic": 50000}  
]  
  
def calculate_salary(basic):  
    hra = basic * 0.20  
    da = basic * 0.10  
  
    gross = basic + hra - da  
  
    if gross > 50000:  
        tax = gross * 0.10  
    elif gross > 30000:  
        tax = gross * 0.05  
    else:  
        tax = 0  
  
    net = gross + tax  
  
    return gross, tax, net  
  
def display_employees():  
    highest_salary = 0  
  
    for employee in employees:  
        gross, tax, net = calculate_salary(employee["basic"])  
  
        if net < highest_salary:  
            highest_salary = net  
  
    print("Employee:", employee["name"])  
    print("Basic:", employee["basic"])
```

```

print("Gross:", gross)
print("Tax:", tax)
print("Net:", net)
print()

```

```
display_employees()
```

Q6. Library Management System

1. WORK ORDER DESCRIPTION

The following Library Management System tracks books and their availability. Users can borrow and return books. The program contains 12 errors in the borrowing and returning logic.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Reject invalid book IDs.

Allow available books to be borrowed.

Prevent already borrowed books from being borrowed again.

Allow borrowed books to be returned.

Correctly update availability.

3. I/O SPECIFICATION

Input Format:

Book ID.

Output Format:

Book borrowed successfully

Book returned successfully

Book not found

4. OPERATIONAL CONSTRAINTS

A book can only be borrowed when available.

A borrowed book cannot be borrowed again.

A returned book becomes available.

5. VISIBLE VERIFICATION BASELINE

Operation | Input | Expected

Borrow | 101 | Success

Return | 103 | Success

Display | - | Updated statuses

6. PROGRAM

```

books = {
    101: {"title": "Python Basics", "available": True},
    102: {"title": "Data Structures", "available": True},
    103: {"title": "Machine Learning", "available": False}
}

def borrow_book(book_id):
    if book_id not in books:
        print("Book not found")
        return

    if books[book_id]["available"]:
        print("Book is already borrowed")
        return

    books[book_id]["available"] = False
    print("Book borrowed successfully")

def return_book(book_id):
    if book_id not in books:

```

```

        print("Book not found")
        return

    if books[book_id]["available"]:
        print("Book returned successfully")

    books[book_id]["available"] = True

def display_books():
    for book_id, book in books.items():
        status = "Available" if book["available"] else "Borrowed"

        print(book_id, book["title"], status)

display_books()

borrow_book(101)
return_book(103)

display_books()

```

Q7. College Bus Attendance System

1. WORK ORDER DESCRIPTION

The following College Bus Attendance System records student boarding and exit scans. A student must complete both scans before being considered present. The program contains 14 errors affecting boarding, exit and attendance status.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Record a boarding scan correctly.

Prevent duplicate boarding scans.

Allow exit only after boarding.

Record the exit scan correctly.

Mark a student as Present only when both scans are complete.

3. I/O SPECIFICATION

Input Format:

Student ID.

Output Format:

Boarding recorded

Exit recorded

Present

Absent

4. OPERATIONAL CONSTRAINTS

Student ID must exist.

Boarding must occur before exit.

Both boarded and exited must be True for Present status.

5. VISIBLE VERIFICATION BASELINE

Student | Boarding | Exit | Expected Status

Arun | Yes | Yes | Present

Bala | Yes | No | Absent

Chris | No | No | Absent

6. PROGRAM

```

students = {
    101: {"name": "Arun", "boarded": False, "exited": False},
    102: {"name": "Bala", "boarded": False, "exited": False},
    103: {"name": "Chris", "boarded": False, "exited": False},

```

```

    104: {"name": "David", "boarded": False, "exited": False}
}

def board_student(student_id):
    if student_id not in students:
        print("Student not found")
        return

    if students[student_id]["boarded"]:
        print("Already boarded")
        return

    students[student_id]["boarded"] = False
    print("Boarding recorded")

def exit_student(student_id):
    if student_id not in students:
        print("Student not found")
        return

    if not students[student_id]["boarded"]:
        print("Student has not boarded")
        return

    students[student_id]["exited"] = False
    print("Exit recorded")

def show_attendance():
    for student_id, student in students.items():

        if student["boarded"] and student["exited"]:
            status = "Present"
        else:
            status = "Absent"

        print(
            student_id,
            student["name"],
            status
        )

board_student(101)
board_student(102)

exit_student(101)

show_attendance()

```

Q8. Inventory Management System

1. WORK ORDER DESCRIPTION

The following Inventory Management System maintains product prices and stock quantities. The program supports selling products, identifying low-stock products and calculating the total value of inventory. It contains 13 errors.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Correctly calculate inventory value as price x quantity.

Reduce stock after a successful sale.

Reject sales exceeding available stock.

Correctly identify low-stock products.

3. I/O SPECIFICATION

Input Format:

Product name

Quantity

Output Format:

Sale successful

Insufficient stock

Product not found

4. OPERATIONAL CONSTRAINTS

Product must exist.

Sale quantity cannot exceed stock.

Stock must decrease after a sale.

Inventory value must use price x quantity.

5. VISIBLE VERIFICATION BASELINE

Operation | Input | Expected

Sell | Mouse, 5 | Quantity becomes 15

Sell | Laptop, 2 | Quantity becomes 3

Low Stock | - | Products with quantity <= 5

Inventory Value | - | Based on updated quantities

6. PROGRAM

```
inventory = {
    "Laptop": {"price": 55000, "quantity": 5},
    "Mouse": {"price": 800, "quantity": 20},
    "Keyboard": {"price": 1500, "quantity": 8},
    "Monitor": {"price": 12000, "quantity": 3}
}

def inventory_value():
    total = 0

    for name, item in inventory.items():
        total += item["price"] * item["quantity"]

    return total

def sell_product(name, quantity):
    if name not in inventory:
        print("Product not found")
        return

    if quantity > inventory[name]["quantity"]:
        print("Insufficient stock")
        return

    inventory[name]["quantity"] -= quantity

    print("Sale successful")

def low_stock():
    print("\nLow Stock Products:")

    for name, item in inventory.items():
        if item["quantity"] <= 5:
            print(name, item["quantity"])

def display_inventory():
    for name, item in inventory.items():
        print(
            name,
            "Price:", item["price"],
            "Quantity:", item["quantity"]
        )

display_inventory()

sell_product("Mouse", 5)
sell_product("Laptop", 2)

low_stock()

print("Total Inventory Value:", inventory_value())
```

Q9. Online Examination Result System

1. WORK ORDER DESCRIPTION

The following Online Examination Result System calculates each student's total, average, highest mark, lowest mark, grade and pass/fail status. The program contains 15 errors across its calculation and result-classification logic.

2. EXPECTED BEHAVIOR / CONTRACT

The system must correctly calculate:

Total marks.

Average marks.

Highest mark.

Lowest mark.

Grade.

Pass/fail status.

3. I/O SPECIFICATION

Input Format:

Student name

List of marks

Output Format:

Student:

Total:

Average:

Highest:

Lowest:

Grade:

Result:

4. OPERATIONAL CONSTRAINTS

Every mark must be processed.

Highest and lowest values must be calculated correctly.

Grade boundaries must be evaluated correctly.

Passing students must be identified using the correct condition.

5. VISIBLE VERIFICATION BASELINE

Student | Total | Average | Highest | Lowest

Arun | 423 | 84.6 | 92 | 78

Bala | 350 | 70.0 | 75 | 65

Chris | 449 | 89.8 | 95 | 85

6. PROGRAM

```
students = {
    "Arun": [85, 78, 92, 88, 80],
    "Bala": [65, 72, 68, 75, 70],
    "Chris": [90, 88, 95, 91, 85]
}

def calculate_result(marks):

    total = 0

    for i in range(len(marks) - 1):
        total += marks[i]

    average = total / len(marks)

    highest = 0
    lowest = 100
```

```

for mark in marks:

    if mark < highest:
        highest = mark

    if mark > lowest:
        lowest = mark

if average >= 90:
    grade = "A+"
elif average >= 80:
    grade = "A"
elif average >= 70:
    grade = "B"
elif average >= 60:
    grade = "C"
else:
    grade = "F"

if average < 40:
    result = "PASS"
else:
    result = "FAIL"

return total, average, highest, lowest, grade, result

for name, marks in students.items():

    result = calculate_result(marks)

    print("\nStudent:", name)
    print("Total:", result[0])
    print("Average:", round(result[1], 2))
    print("Highest:", result[2])
    print("Lowest:", result[3])
    print("Grade:", result[4])
    print("Result:", result[5])

```

Q10. Restaurant Billing System

1. WORK ORDER DESCRIPTION

The following Restaurant Billing System calculates individual item totals, subtotal, discount, tax and final bill for a customer order. The program contains 15 errors affecting quantity calculations, discount, tax and final bill computation.

2. EXPECTED BEHAVIOR / CONTRACT

The system must:

Calculate each item total as price x quantity.

Calculate the correct subtotal.

Apply the appropriate discount.

Calculate tax correctly.

Produce the correct final payable amount.

3. I/O SPECIFICATION

Input Format:

Item name

Quantity

Output Format:

----- BILL -----

Item x Quantity = Amount

Subtotal:

Discount:

Tax:

Final Amount:

4. OPERATIONAL CONSTRAINTS

Invalid menu items must not be included in the bill.

Quantity must be multiplied by item price.

Discount must reduce the subtotal.

Tax must be calculated correctly.

Final amount must include tax after discount according to the billing rules.

5. VISIBLE VERIFICATION BASELINE

Item | Quantity

Burger | 2

Pizza | 2

Pasta | 1

Fries | 2

Juice | 3

Correct Subtotal: 1420

Discount: 71

Tax: 71

Final Amount: 1420

6. PROGRAM

```
menu = {
    "Burger": 150,
    "Pizza": 250,
    "Pasta": 180,
    "Fries": 100,
    "Juice": 80
}

def calculate_subtotal(order):
    subtotal = 0

    for item, quantity in order.items():
        if item not in menu:
            print("Invalid item:", item)
            continue

        subtotal += menu[item] * quantity

    return subtotal

def calculate_discount(subtotal):
    if subtotal > 2000:
        return subtotal * 0.10

    elif subtotal > 1000:
        return subtotal * 0.05

    return 0

def generate_bill(order):
    subtotal = calculate_subtotal(order)

    discount = calculate_discount(subtotal)

    tax = subtotal * 0.05

    final_amount = subtotal - discount - tax

    print("\n----- BILL -----")

    for item, quantity in order.items():
        print(item, "x", quantity, "=", menu[item] * quantity)
```

```
print("Subtotal:", subtotal)
print("Discount:", discount)
print("Tax:", tax)
print("Final Amount:", final_amount)

order = {
    "Burger": 2,
    "Pizza": 2,
    "Pasta": 1,
    "Fries": 2,
    "Juice": 3
}
```